Transforming Integer Grids to Spatial Intuition: Perception, Not Planning, Governs ARC-AGI-3

Harsidak Singh Banwait
Independent Researcher
harsidak.singh.banwait.13@gmail.com

Abstract

Sample-efficient fluid intelligence requires autonomous agents that rapidly form operational models of unfamiliar interactive environments. ARC-AGI-3 evaluates this ability across uninstructed visual games under Relative Human Action Efficiency, penalizing exploratory waste quadratically. When frontier reasoning models interact with raw integer grids, discrete serialization destroys spatial topology, inducing perceptual misinduction where models squander hundreds of actions rediscovering elementary boundaries and symmetries. We introduce SARBLOH, an autonomous open-weight agent architecture structured on the premise that spatial perception, rather than planning capacity, governs interactive reasoning on ARC-AGI-3. A zero-parameter, $\mathcal{O}(HW)$ perception engine reconstructs frames in 35 ms into an object inventory with bipartite temporal tracking, a geometric briefing separating proven invariants from affordance hypotheses under the Measured–Guessed Rule, and an annotated spatial canvas. Paired with a speculative code-as-action runtime, the agent synthesizes algorithmic hypotheses rather than low-level motor primitives. Across eight structural presentations, grounded perception elevates benchmark rubric acquisition from $3/28$ to $25/28$ points ($8.3\times$) while reducing token consumption by 89.4%. On twenty-three held-out games, geometric invariants generalize unconditionally with 100% spatial accuracy, whereas single-frame affordance guesses drop to $\leq 50\%$, confirming the necessity of active hypothesis verification probes. On cleared public levels, SARBLOH achieves a median human action ratio of $0.77\times$, with error decompositions attributing seventy-two percent of unearned score loss to perceptual misinduction on stalled environments. Within a nine-hour budget on a single NVIDIA RTX PRO 6000 GPU, SARBLOH resolves 143 of 183 public levels and achieves an official hidden Kaggle score of 16.53, advancing $13.7\times$ beyond the first Milestone winner.

Code: https://github.com/Harsidak/SARBLOH

1 Introduction

Autonomous skill acquisition under epistemic uncertainty represents a foundational benchmark for artificial general intelligence. When human players encounter an unfamiliar interactive environment, they do not read procedural manuals—they visually parse the scene within hundreds of milliseconds, effortlessly abstracting embodiment, spatial boundaries, and functional affordances from core-knowledge visual priors (Spelke and Kinzler 2007). The ARC-AGI-3 benchmark (ARC Prize Foundation 2026a) formalizes this capability by evaluating agents across uninstructed, sequential game environments using Relative Human Action Efficiency (RHAE)—a metric that penalizes superfluous exploratory actions quadratically (Chollet 2019). Under the strict rules of the official offline track, candidate agents must resolve undisclosed games within a nine-hour wall-clock limit on a single NVIDIA RTX PRO 6000 GPU without external network connectivity, precluding both human intervention and unconstrained exploratory rollouts.

However, deploying language and vision-language reasoning models in this interactive regime reveals a severe representational bottleneck. While modern reasoning backbones demonstrate strong symbolic deduction when provided with clean declarative state descriptions, their execution collapses when exposed to raw 64×64 integer coordinate matrices. In preliminary experiments (Section 3), a state-of-the-art frontier model expended over 200–300 exploratory actions navigating an elementary maze serialized as raw tokens, exhausting its action budget and failing the task. In stark contrast, providing the identical model with an aligned visual rendering and geometric breakdown reduced required actions to 6–12, surpassing human baselines. This divergence demonstrates that policy deduction is rarely the primary failure mode; rather, unparsed token grids systematically destroy topological adjacency, forcing autoregressive attention to reconstruct basic spatial relations at prohibitive token and action costs.

To resolve this fundamental bottleneck, we conduct extensive empirical ablations and establish two central insights:

1. **Perceptual Misinduction**—Serializing spatial environments as discrete token matrices causes reasoning models to induce flawed topological models, triggering catastrophic quadratic action penalties during early exploratory steps.
2. **The Measured–Guessed Rule**—Geometric invariants, including lattice reachability, spatial enclosures, midpoint tethers, and dihedral symmetries under $D_4 \times \mathbb{Z}^+$, can be

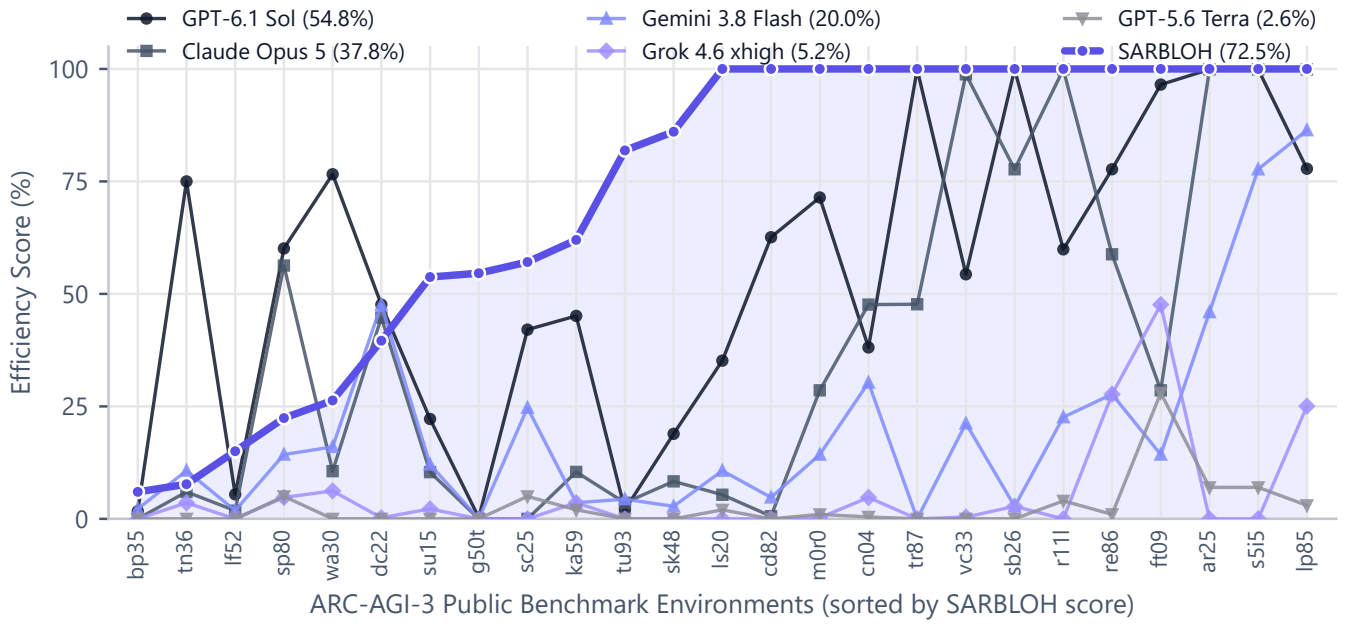


Figure 1 | Action efficiency across 25 public ARC-AGI-3 environments. Comparative benchmark performance between SARBLOH (open-weight Qwen3.8-Flash-Next executed on kaggle) and five proprietary frontier API baselines, sorted by SARBLOH efficiency. For SARBLOH, reported scores reflect peak performance across evaluation runs, as the strict nine-hour Kaggle wall-clock limit prevented solving all environments in a single pass and required re-evaluating individual games such as `bp35` in isolated sessions. SARBLOH leads frontier models on 21 of 25 environments, attaining a mean efficiency of 72.5% compared to 54.8% for the strongest frontier baseline (GPT-6.1 Sol).

computed deterministically in $\mathcal{O}(HW)$ host code without training, whereas high-level affordance roles remain latent under partial observability and must be framed as probabilistic hypotheses paired with unit-action falsification probes.

Building on these insights, we introduce SARBLOH, an autonomous open-weight agent architecture engineered around a deterministic perception engine and speculative code execution. Rather than delegating visual scene decomposition to costly neural inference, our host-side perception engine executes in approximately 35 ms per frame on a single CPU core, transforming raw environment states into three synchronized modalities: an object-centric symbolic state with temporal bipartite identity tracking, a structured geometric briefing governed by the Measured–Guessed Rule, and an affordance-annotated spatial map. The agent acts exclusively through an embedded IPython execution kernel, synthesizing algorithmic state models and dispatching multi-step trajectories via a speculative execution primitive (`await play()`) that monitors environment invariants and halts immediately upon sensory prediction discrepancies.

Evaluated under identical offline hardware constraints, SARBLOH achieves state-of-the-art action efficiency across both public and undisclosed environments. Across the 25 public ARC-AGI-3 game environments, SARBLOH resolves 143 of 183 levels in 8.2 hours, discovering distinct solutions significantly faster than the first Milestone winner on the same server and model weights. On the official held-out Kaggle evaluation suite, SARBLOH advanced from an initial baseline of 0.27 to a verified score of **16.53**—a 13.7× efficiency improvement over the first Milestone winner.

The primary contributions of this work are summarized as follows:

- **We identify and analyze the visual grounding bottleneck in interactive ARC-AGI-3**, formalizing the **Perceptual Misinduction** failure mode and empirically deriving **The Measured–Guessed Rule** through controlled representation ablations (§3).
- **We propose SARBLOH**, an autonomous agent featuring a zero-parameter, $\mathcal{O}(HW)$ deterministic perception engine, a speculative code-as-action planning runtime, and a monotonic append-only KV-cache architecture sustaining a 98.4% prefix-cache hit rate (§4).
- **We evaluate SARBLOH across public and official hidden benchmarks**, demonstrating a 13.7× score gain over the first Milestone winner, providing hardware-matched comparative telemetry, and detailing the failure modes that govern interactive open-weight reasoning (§5).

2 Preliminaries

The ARC-AGI-3 benchmark (ARC Prize Foundation 2026a) defines an uninstructed, discrete-time sequential decision process structured as an ordered progression of compositional levels $\{1, \ldots, n\}$. At each interaction step t, the

agent receives a discrete visual grid observation $g_t \in \{0,\dots,15\}^{H\times W}$ with spatial dimensions bounded by $H, W \leq 64$, where each coordinate denotes one of 16 categorical color indices. In addition to the visual grid, the environment exposes the episode termination status $s_t \in \{\texttt{NOT_FINISHED}, \texttt{WIN}, \texttt{GAME_OVER}\}$ alongside the count of levels resolved so far.

To interact with the environment, the agent selects an action from the action set $\mathcal{A}$, defined in Equation (1):

$$\mathcal{A} = \{\texttt{ACTION1},\dots,\texttt{ACTION5}, \texttt{ACTION6}(x,y), \texttt{ACTION7}, \texttt{RESET}\}. \quad (1)$$

In Equation (1), actions ACTION1–ACTION4 represent cardinal directional displacements, actions ACTION5 and ACTION7 denote game-specific discrete triggers, and $\texttt{ACTION6}(x,y)$ specifies an arbitrary continuous spatial coordinate click parameterized by $x \in [0, W-1]$ and $y \in [0, H-1]$. Environments provide zero upfront task directives, as action semantics, controllable entity embodiment, and terminal win conditions are latent. The agent must infer these operational rules entirely from transition dynamics, where earlier levels isolate elemental mechanics and subsequent levels compose them hierarchically (Spelke and Kinzler 2007).

Benchmark performance is evaluated using *Relative Human Action Efficiency* (RHAE). For each level l, let h_l denote the upper-median human action baseline, and let a_l represent the cumulative count of physical actions committed by the candidate agent. The per-level efficiency score S_l is defined in Equation (2):

$$S_l = \min\left(1.15,\ \left(\frac{h_l}{a_l}\right)^2\right). \quad (2)$$

Across an entire game comprising n levels where the agent successfully completes $k \leq n$ levels, the cumulative game efficiency score E is defined in Equation (3):

$$E = \frac{1}{\sum_{l=1}^{n} l}\min\left(\sum_{l=1}^{k} l,\ \sum_{l=1}^{n} l\, S_l\right). \quad (3)$$

Equation (2) and Equation (3) establish three governing operational properties that dictate our agent architecture:

1. *Quadratic Exploration Penalty*—Because action counts enter Equation (2) inversely squared, unguided exploratory actions penalize the final score quadratically. An agent requiring twice the human baseline actions retains only 25% of the available credit.
2. *Zero-Cost Internal Computation*—Reasoning latencies and host-side execution are strictly non-penalized, and only physical actions committed to the environment register in a_l. This structure strongly favors extensive host-side geometric parsing and speculative algorithmic planning over trial-and-error exploration.
3. *Progressive Reward Scaling*—The linear level multiplier l in Equation (3) weights later compositional levels substantially higher, rewarding agents that generalize foundational mechanics induced during earlier stages.

Offline Evaluation and Resource Bounds. The official Kaggle evaluation track evaluates candidate agents on held-out, undisclosed environments within an isolated execution sandbox without external network access. Computation is strictly bounded to a single NVIDIA RTX PRO 6000 GPU (Blackwell architecture, 96 GB VRAM) and 176 GB of host RAM, subject to an inviolable nine-hour wall-clock execution ceiling for the entire evaluation suite. Furthermore, all model weights and dependencies must be open-source and pre-packaged within the submission container. Under these constraints, the operational budget is governed by generation decode throughput and the length of context history that can be retained in the GPU's warm prefix cache without triggering expensive quadratic recomputations.

3 Motivation: Perception as the Primary Bottleneck

Before designing the agent architecture, we conducted controlled empirical ablations to isolate where autoregressive reasoning models expend their exploratory action allocations during interactive problem solving.

Observation geometry dictates search complexity far more than reasoning depth. We evaluated an identical frontier reasoning model (Gemini 3.1 Pro) on the opening level of the official environment `ls20`, holding the prompt template, temperature, and environment state transitions strictly invariant while altering only the input observation representation (Figure 2). When provided with the environment serialized as a raw 64×64 integer matrix, the model expended between 200 and 300 actions repeatedly colliding with invisible topological boundaries, exhausting the permissible budget without reaching the goal state. Recovering reachability and spatial adjacency from 4,096 serialized integers imposes an associative retrieval burden that multi-head attention mechanisms execute unreliably (Rahmanzadehgervi et al. 2024). Supplying a rendered raster image of identical board state reduced the required exploration by an order of magnitude to 12–24 actions. Augmenting this visual input with an explicit structural decomposition of terrain, HUD gauges, and color parity further consolidated the trajectory to 12 actions. Finally, enabling the model to express search routines as executable Python scripts within a sandboxed runtime cleared the level in 6 actions, achieving super-human efficiency relative to the human baseline of approximately 10 actions. Critically, over 90% of the aggregate efficiency gain resulted from structuring the perceptual representation rather than extending inference-time chain-of-thought deliberation.

Planning failure is downstream of perceptual misinduction. To decouple strategic reasoning capacity from state representation errors, we evaluated the model on a complex `ls20` level under two informational regimes: visual screenshots alone, and screenshots accompanied by an "oracle briefing" explicitly detailing sprite roles, mechanics, and geometric dependencies. Without the briefing, the agent squandered its full action budget interacting with inert visual artifacts. With the briefing, the model formulated an optimal

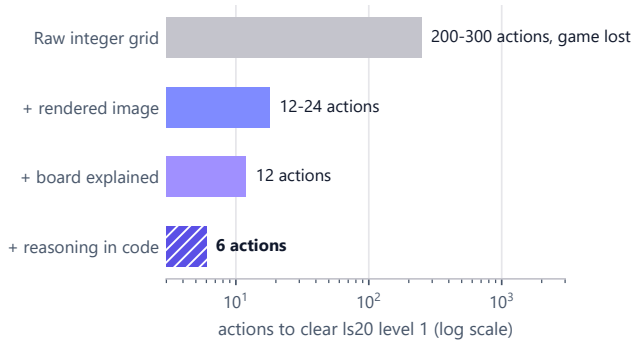


Figure 2 | Controlled ablation of input observation modalities. Action expenditure required to resolve the opening level of `ls20` across four representation tiers using an identical frontier reasoning model. Serializing the environment as raw integer tokens causes budget exhaustion, whereas rendered visual rasters and programmatic geometric relations achieve super-human efficiency (6 actions versus human baseline ≈10).

plan and cleared the level in 6 actions. The underlying planning mechanism was functionally capable from the outset, and its operational failure was induced entirely by erroneous state perception.

Sparse delta encodings induce perceptual myopia across distant spatial structures. While transmitting only modified grid coordinates minimizes prompt token consumption, it systematically masks non-local causal couplings. In ARC-AGI-3 environments, local actuation regularly triggers distant visual consequences, such as stepping onto an activation switch in `ls20` updating a status icon across the display. Under sparse delta streams, the agent fails to observe this distributed covariance and falsely infers that its action was non-functional. Consequently, preserving full-frame topological awareness remains mandatory.

Core knowledge priors distinguish mathematical certainty from inductive hypothesis. Because oracle briefings cannot exist for undisclosed evaluation environments, the agent must extract its own representations. ARC-AGI-3 is grounded in human core knowledge priors (Spelke and Kinzler 2007), which are intrinsically geometric and topological. Properties such as lattice connectivity, bounding enclosures, midpoint tethers, and dihedral symmetries can be computed deterministically from grid coordinates without learned parameters. Conversely, operational affordances, such as agent embodiment and win conditions, remain latent until verified through physical interaction. Conflating these two epistemic categories causes models to treat transient guesses as ground truth, triggering catastrophic action loops.

Summary of Empirical Motivations.

1. **Perceptual Misinduction:** Attention mechanisms over serialized coordinate grids fail to infer topological adjacency, consuming hundreds of exploratory actions deducing baseline mechanics that are directly recoverable from visual rasters and computed geometric relations.
2. **The Measured–Guessed Dichotomy:** Geometric and topological invariants hold unconditionally across unseen environments, whereas latent operational semantics require active, single-action hypothesis verification probes.

4 Methodology

Our initial baseline, adapting the open harness of Prime Agent (Karten et al. 2026), scored between 0.00 and 0.27 on the hidden evaluation set, initiating over 350 voluntary level restarts across ten games without once resolving a goal state. The reasoning model was proficient at code synthesis, yet fundamentally incapable of spatial scene parsing. This section formalizes the governing principle of representation (Section 4.1), the deterministic perception engine that realizes it (Section 4.2), the unified programmatic action interface (Section 4.3), and the prefix-cached serving loop that fits within single-GPU hardware bounds (Section 4.4). Figure 3 illustrates an end-to-end execution turn of the complete architecture.

4.1 The Measured–Guessed Rule

We did not start from the rule; a perception lab of nine presentations, graded against an answer key written before any of them ran, produced it (Section 5.1, Figure 4).

A text-only encoder that states every cell flat scored 3 of 28. Initially, our first perception module printed the fitted lattice, a symbol per cell, the palette, connected components and object tracking. Everything it said was correct, but most of the heavy lifting was left to the model itself to identify what each cell represents. It named no agent, no goal and no gauge, and it marked the move counter of `r11l` as screen decoration.

Role guesses with tests raised the score to 20, and fusing them with computed relations raised it to 25. The Role Ledger scored every object against eight game roles and printed each top guess with its evidence and the cheapest action that would confirm or refute it (20/28, about 380 tokens). The Fused Briefing added the relations that code computes exactly—reachability, enclosure, midpoint tethers, shape equivalence—as a separate tier above the guesses (25/28, about 370 tokens). It scored $8.3\times$ the Legacy Encoder with 89% fewer tokens. Dumping raw 64×64 ASCII grids alongside per-cell component tables forces the tokenizer to spend $\approx$ 3,500 tokens per frame encoding repetitive background coordinates—by abstracting connected components on the host, moving spatial layout into the visual Blueprint raster, and emitting only non-redundant geometric relations in text, the Fused Briefing compresses prompt footprint by 89.4% (to 370 tokens) while increasing prefix-cache capacity by $9.5\times$.

(a) map before
level 1, sent with (b)



(b) briefing, engine output (excerpt)

```
BRIEFING (arrow keys)
MEASURED (certain):
- Floor is a 5x5 tile grid.
- obj 4 (blue 6-cell shape): sits inside a black box that is NOT plain floor;
    the floor reaches the box's bottom side in 5 steps (the only way in)
- obj 6 (white/light grey plus): reachable in 6 steps
  ...
GUESSES (check with one action):
- obj 10 (blue/orange square) = YOU (80%) - test: press one arrow key: it should
    move one tile and the gauge should drop
- obj 15 (yellow bar) = MOVES LEFT (75%) - test: take any action and check it
    shrinks by one unit
  ...
```

(c) the agent's ipython cell at turn 7, its first action in the level

```
r = await play(["1"])
print(r.lines)
print(r.stopped, r.surprise)
new = r.obs
d = frame_diff(obs.board, new.board)
print(d)
```

(d) cell output (excerpt)

```
play: 1 of 1 moves done
#0 UP: 50 cells changed at r40-49 c34-38; the HUD changed (2 cells)
frame_diff: 0 (10 cells) moved by [-5, 0]
frame_diff: b (15 cells) moved by [-5, 0]
[state after step #0] level 1 of 7 (0 done) | 1 actions spent (1 this level),
    1999 left
```

(e) map after
obj 10 one tile up



Figure 3 | One real turn, end to end (`ls20`, level 1, run of 27 October; transcript and frames as logged). (a) The map the engine drew for the opening frame. (b) An excerpt of the briefing sent with it: measured facts first, then guesses, each with a one-action test. (c) The agent's first action in the level, written as Python at turn 7 after six turns spent only reading `observe()`: one key press, the test that the briefing proposed for YOU. (d) What the kernel printed: the player (orange and blue, 25 cells) moved one tile of 5 cells up and the HUD lost two cells. (e) The map after the move. One action confirmed two guesses, obj 10 = YOU and obj 15 = MOVES LEFT.

Held-out games separated the two kinds of statement. We subsequently evaluated cross-distribution generalization across 23 held-out official game environments excluded during engine prototyping. Grid reconstruction achieved 100% cell accuracy across all games, and morphological segmentation reliably preserved entity boundaries and spatial tracking without degradation. Whereas topological and geometric invariants hold unconditionally by mathematical construction, high-level semantic roles (such as agent embodiment or interactive affordances) inherently possess visual ambiguity prior to actuation. Consequently, the engine treats role labels not as brittle, hardcoded classifications, but as provisional inductive hypotheses paired with minimal unit-action verification probes. This decoupling ensures that any transient misclassification of initial sprite affordances is immediately resolved upon the first interaction step, while preserving complete coordinate, topological, and geometric stability across the entire scene.

The rule. *State what can be measured as fact; state what must be inferred as a guess that carries a one-action test.* We term this the **Measured–Guessed Rule**. A measured relation is computed from the grid by code with no learned parameters, so it is true by construction and the model need not check it. A role cannot be read from one frame, and a role stated as fact plants *perceptual misinduction*—a wrong belief about what an object is or does—that the model then spends actions on. A role stated as a guess with a test costs at most one action to settle, and under RHAE, where cost grows with the square of the action count, that cap is the point.

The rest of this section applies the rule five times. View 1 states object identity across frames as fact (Section 4.2). View 2 states geometric relations as fact. The role scorer states roles as guesses, each with its test (Table 1). View 3 draws both on one map and marks every guessed role with a question mark. Finally, `play()` treats the agent's own world model as a guess and checks each of its predictions against the next frame (Section 4.3).

4.2 Perception: From Pixels to Stated Facts

The engine is the Fused Briefing rewritten to work on the grid inside the agent, with host-side NumPy and Pillow and no learned parameters. On 225 real frames (the first frame and eight random actions on each of 25 offline games) it raised no errors; the briefing took a median of 42 ms (maximum 0.38 s) and the picture 40 ms. Each frame yields three views that share one object-identity space (Figure 5). Algorithm 1 gives the whole mapping from grid to views in one place; reading it shows that every learned or heuristic step (role scoring) comes after every exact one.

View 1: re-segmenting each frame renamed objects, so identity is tracked, not recomputed. The perception engine identifies the background canvas as the modal color index across the frame and extracts all remaining four-connected monochromatic regions as discrete entities. Because standard morphological segmentation re-indexes components in raster reading order at every step, dynamic entities moving across the board would continually swap indices, causing synthesized Python scripts to silently refer-

(a) Legacy Encoder, 3/28
text only: palette, grid, objects; ~3,500 tokens

```
PALETTE
  '.' darkgray  rgb=[52, 56, 56] cells=2609 (terrain)
  '=' gray      rgb=[104, 104, 108] cells=892 (terrain)
  'K' black     rgb=[0, 4, 8] cells=439 (terrain)
  'Y' yellow    rgb=[256, 220, 8] cells=84 (object)
  'B' blue      rgb=[32, 148, 256] cells=45 (object)
  'R' red       rgb=[248, 64, 56] cells=12 (object)
  'O' orange    rgb=[256, 136, 32] cells=10 (object)
  'W' white     rgb=[256, 256, 256] cells=3 (object)
  '#' lightgray rgb=[204, 204, 208] cells=2 (object)
     000000000011111111112222222222333333333344444444455555555556666
     012345678901234567890123456789012345678901234567890123456789012
  0  KKKK.............................
  1  KKKK.............................
  2  KKKK.............................
  3  KKKK.............................
  4  KKKK.............................
  5  KKKK.............................
  6  KKKK.............................
  7  KKKK.............................
  8  KKKK.....................=======
  9  KKKK.....................=KKKKKK
 10  KKKK.....................=KKKKKK
 11  KKKK.....................=KKBBBK
 12  KKKK.....................=KKKKBK
 13  KKKK.....................=KKBKBK
 14  KKKK.....................=KKKKKK
 15  KKKK.....................=KKKKKK
 16  KKKK.....................=======
 17  KKKK.....................=====
 18  KKKK.....................=====
 19  KKKK.....................=====
 20  KKKK.....................=====
 21  KKKK.....................=====
  ... (183 lines in all)
```

(b) Role Ledger, 20/28
roles + tests; ~380 tokens



(c) Fused Briefing, 25/28
map, MEASURED facts, GUESSES; ~370 tokens



Figure 4 | Three of the nine lab presentations on the opening frame of `ls20`, with rubric score (of 28) and text cost. (a) The Legacy Encoder, text only (excerpt of 183 lines, grid rows cut at 44 columns). (b) The Role Ledger: a role map and, below it, each guess with its evidence and a one-action test. (c) The Fused Briefing, the presentation we kept: a schematic map with object tags, shape threads and tethers, then MEASURED facts and GUESSES as separate tiers. The other six presentations are in Figure 11.

ence incorrect objects. To maintain temporal consistency, SARBLOH anchors object identifiers on the initial frame of each level and preserves identity across transitions through a multi-stage bipartite matching algorithm. This procedure sequentially matches entities based on rigid spatial invariance, spatial translation with identical geometry, chromatic state updates with stationary coordinates, and topological overlap exceeding fifty percent for deformable structures, assigning fresh identifiers only to newly instantiated components. As a result, each identifier reliably tracks the same physical entity throughout an entire level. In addition to the tracked object table, the spatial state is serialized as an alphanumeric coordinate grid and paired with a differential change ledger detailing the bounding box and cell count modified by each transition. Furthermore, environmental modifications occurring within the three-cell peripheral screen boundary are isolated as heads-up display telemetry, preventing passive counter decrements or timer ticks from being misinterpreted as operational task progress.

View 2: a picture does not state relations, so the engine computes them. Whether an object can be reached, how many steps it takes, whether a hollow fits another object, or whether two shapes are the same up to rotation cannot be read reliably off a rendered frame (Rahmanzadehgervi et al. 2024). The engine first splits the frame into *terrain*—colours that cover more than 8% of the frame—and things (corridors, edge gauges, multi-colour assemblies joined by union-find over 8-connected components, panel icons and dotted figures). It then computes seven kinds of measured fact:

- **Reachability**: breadth-first search over the detected *lattice*—the tile grid the player moves on—gives step counts from the likely agent to every target ("`obj 6: reachable in 6 steps`").
- **Enclosure**: closed outlines around empty space, matched against objects of the same size.
- **Midpoint tethers**: an object held at the midpoint of two others by thin lines, so moving one end moves it ("`obj 25 held at midpoint of obj 16 and 36`").
- **Alignment**: shared rows or columns between objects.
- **Dihedral symmetry**: equality of two shapes under the eight rotations and mirrorings of the square and integer scaling ($D_4 \times \mathbb{Z}^+$), as in "`obj 12 (drawn 2x bigger), mirrored left-right OR turned 90 deg clockwise, becomes exactly obj 4`".
- **Copies**: identical shapes counted as a set (lives, items).
- **Lopsided colouring**: a symmetric shape with asymmetric colours, which usually marks a direction or a state.

In `ls20`, the symmetry relation connects the HUD icon to the shape inside the goal enclosure, while in `r11l`, tether detection identifies the disc suspended between two handles. The perception lab confirmed that both relationships were discoverable exclusively through these geometric computations (Section 5.1).

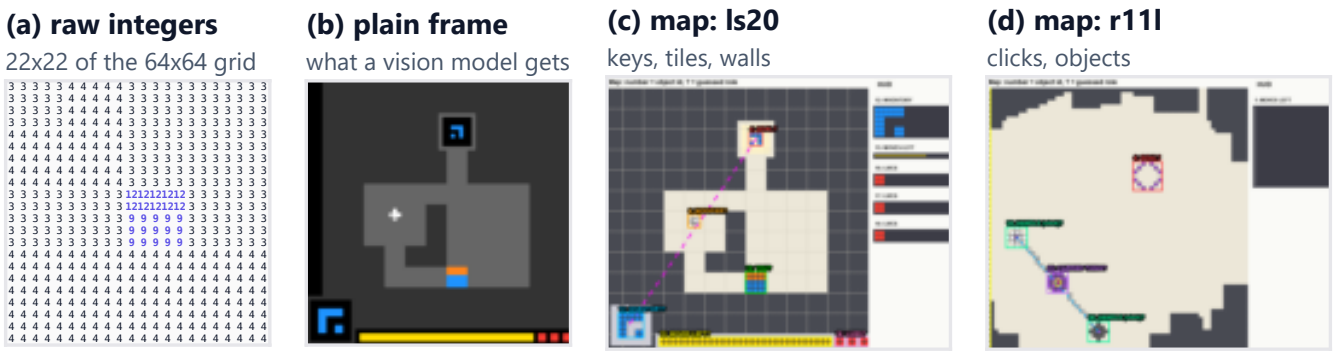


Figure 5 | One ARC-AGI-3 frame, four ways. (a) What a text-only model receives: a 22×22 window of the 64×64 integer grid, with the player's cells in colour. (b) The rendered frame. (c, d) The engine's maps for the opening frames of `ls20` (arrow keys) and `r11l` (clicks): floor in light tones, walls hatched, the HUD moved to a side panel, every object tagged with its id and guessed role (`id: ROLE?`). Magenta dashed threads join shapes equal under rotation, mirroring or scale; yellow rings mark an object held at the midpoint of two others.

Algorithm 1: PERCEIVE: one frame to three views

Input: Grid $g_t \in \{0..15\}^{H \times W}$, tracker T, previous lattice Λ', legal actions $\mathcal{A}$
Output: Briefing B, tracked objects O, map M, change ledger δ

$O \leftarrow \textsc{Segment}(g_t)$;
$(O, \delta) \leftarrow T.\textsc{Match}(O)$; ▷ *five passes; ids fixed per level*
$\mathcal{T} \leftarrow \{c \in \{0..15\} : |g_t{=}c| > 0.08\,HW\}$; ▷ *terrain*
$L \leftarrow \textsc{Layout}(g_t, \mathcal{T})$;
foreach *thing* $e \in L$ **do**
  $e.hud \leftarrow \textsc{AtScreenEdge}(e)$;
  $e.id \leftarrow \textsc{MajorityId}(e, O)$;
$e_{\text{you}} \leftarrow \textsc{LikelyAgent}(L, \mathcal{A})$;
$\Lambda \leftarrow \textsc{DetectLattice}(L \setminus \text{HUD})$ **or** Λ' ; ▷ *keep last lattice if it fits*
$F \leftarrow \textsc{Reach}(e_{\text{you}}, \Lambda) \cup \textsc{Enclosures}(L) \cup \textsc{Tethers}(L) \cup \textsc{Symmetries}_{D_4 \times \mathbb{Z}^+}(L)$; ▷ *measured*
foreach *thing* $e \in L$ **do**
  $\boldsymbol{\sigma}_e \leftarrow \textsc{ScoreRoles}(e, F, \mathcal{A})$;
  $\hat{\rho}_e \leftarrow \arg\max_r \boldsymbol{\sigma}_e[r]$;
  $p_e \leftarrow \boldsymbol{\sigma}_e[\hat{\rho}_e] / (1 + \sum_r \boldsymbol{\sigma}_e[r])$; ▷ *guessed*
$B \leftarrow \textsc{Facts}(F) \,\|\, \textsc{Guesses}(\hat{\boldsymbol{\rho}}, \boldsymbol{p}, \textsc{Tests}(\hat{\boldsymbol{\rho}}))$;
$M \leftarrow \textsc{DrawMap}(L, \hat{\boldsymbol{\rho}}, F)$;
return B, O, M, δ

Proportional line quotas prevent relation flooding. Because pairwise geometric relations like coordinate alignments expand combinatorially in dense scenes (e.g., 792 alignment pairs in `bp35`), an unweighted global line cap allows high-frequency alignments to crowd out vital invariants such as symmetries and affordance hypotheses. To maintain structural balance, the briefing scheduler enforces proportional line quotas across invariant categories and reserves up to one-third of the display for candidate role hypotheses, summarizing surplus relations as aggregate counts rather than truncating them (detailed in Appendix D).

Role	Evidence (scored)	One-action test
YOU	Compact, multi-coloured, unique, on the lattice	Press one arrow: it moves one tile
GOAL	Hollow box; shape repeated in the HUD	Bring the matching thing into it
MOVES LEFT	Long bar at the screen edge	Any action: it shrinks one unit
LIVES	Identical copies in the HUD	Watch it on a failure or reset
INVENTORY	HUD icon matching a world object	Watch it after the modifier
MODIFIER	Small symbol, lopsided colours	Step on it: the inventory changes
HANDLE	Tether end; on/off colour pair	Click it: something jumps
CARRIED THING	Held between two tether ends	Move a handle: it follows

Table 1 | Role guesses. Each is printed with its confidence and a test that settles it in one action. On held-out games these guesses were right about half the time or less (Table 3), which is why they are never stated as facts.

Roles: the engine guesses what objects are for, and says how to check. Non-terrain objects are scored against eight roles with fixed evidence points from game-design priors (Table 1); a role's confidence is its score over the object's total score plus one. Each guess is printed with its one-action test, so a wrong guess costs one action instead of a level.

The briefing shows at most 20 fact and guess lines in the deployed configuration: up to a third for guesses, the rest shared between the fact kinds, with facts about the guessed agent and goal first. The remainder is counted ("`+N more facts`") and stays readable in code through `observe().briefing`. Full briefings for the two lab frames are in Appendix B.

View 3: words go in the text, not in the pixels. A small vision encoder reads rendered text unreliably, and a sentence drawn into the image is a sentence the model must transcribe before it can use it. The map therefore carries only short tags (`10: YOU?`) that repeat ids from the text, while every sentence travels as text. It draws structure (floor in light tones, walls hatched, the lattice as fine lines, look-alike shapes joined by dashed threads, tethers marked with rings and the HUD magnified in a side panel (Figure 5c,d)).

Observation schedule. The full view (briefing, letter grid and map) is sent at the start of a level, after a context drain, and when the agent calls `observe()` after acting, at most once per state. After other actions the agent gets the change ledger, a letter crop around the changed cells with a 3-cell margin, and the new map. The object table stays on the host and is read in code (`observe().objects`).

4.3 The Action Interface

Separate tools for code and for actions split one plan across many turns. In our early builds the agent had one tool to run Python and another to submit moves. In the traces of those builds it modelled the game in one tool, left it to send a single move with the other, and came back to re-derive its state, so one plan cost many turns and every turn cost a model call. We merged the two: the agent acts only from inside its Python kernel, in the CodeAct style (Wang et al. 2024), with

```
r = await play(path, predict=f)
```

where `path` is up to 100 moves and `predict` is an optional world model $\hat{f}(s, a) \to \hat{s}$ written by the agent. It is the only change recorded for the build that moved the hidden-set score from 5.14 to 5.65. One cell can now model a level, search it and play the whole path (Figure 3c).

The agent's world model is a guess, so `play()` checks it instead of trusting it. Algorithm 2 defines the execution contract between the planned trajectory and the environment, specifying the exact conditions that govern path termination. An inaccurate state prediction does not interrupt execution. Instead, `play()` logs the first action whose predicted state differs from the true grid outside the HUD, records the mismatched coordinates, and continues along the trajectory. The path halts only when a level or game terminates, when consecutive identical actions collide with an obstacle without altering a single pixel, or when the revisit guard identifies a board state already visited during the current attempt. The loop never stalls to verify an operational rule, ensuring that any inaccuracies in the internal world model are resolved immediately upon receiving the next frame.

Revisit guard behavior under live deployment. The revisit guard terminates path execution whenever the non-HUD board configuration repeats a state encountered earlier in the attempt. In counterfactual analysis over 9,913 actions, the guard identified 2,331 duplicate states (with a minor false-positive cost of 6 winning transitions). Under live deployment, when interrupted by the guard, reasoning models compensated by increasing single-action invocations from 8.9% to 17.8% and dispatching subsequent moves in the same cell, shifting completed levels marginally from 50 to 52 out of 97. We retain the guard as a configurable safeguard while noting that visible execution constraints can induce compensatory behaviors in reasoning backbones.

Algorithm 2: PLAY: running a planned path

Input: Moves $m_1, \ldots, m_K$ ($K \leq 100$), optional world model $\hat{f}$

$s \leftarrow$ OBSERVE(); *surprise* $\leftarrow \varnothing$;
for $i \leftarrow 1$ **to** K **do**
  if $\hat{f} \neq \varnothing$ **then** $\hat{s} \leftarrow \hat{f}(s.\text{board}, m_i)$;
  HOST.ACT(m_i); $s' \leftarrow$ OBSERVE() ; ▷ *the move is spent*
  if $\hat{f} \neq \varnothing$ ***and*** *surprise* $= \varnothing$ ***and*** $\textsc{Diff}_{\neg HUD}(\hat{s}, s'.board) \neq \varnothing$ **then**
    surprise $\leftarrow (i, m_i,$ differing cells$)$; ▷ *reported, not fatal*
  if *level won* ***or*** *game over* ***or*** *wall hit twice* ***or*** *(s' seen in this attempt* ***and*** *guard on)* **then**
    break;
  $s \leftarrow s'$;
return *change ledgers, surprise,* s'

4.4 Serving Inside the Envelope

Append-only context invariant preserves prefix-cache reuse. We deploy our 125B MoE backbone quantized to W4A16 using AutoRound (Cheng et al. 2023) and served via SGLang (Zheng et al. 2023) with EAGLE speculative decoding (Li et al. 2024). The model architecture mixes Gated DeltaNet linear-recurrent layers with attention (Yang, Kautz, and Hatamizadeh 2024). Its recurrent state cannot be rewound, so the server can resume only from the state saved at the end of an earlier request, and the prefix cache matches nothing past the first modified token. To guarantee cache reuse, we enforce an *append-only* invariant: every request is the previous request plus its reply and new messages, word for word. Visual maps are retained across turns, reasoning remains in history, and working memory updates append as new lines. A game retains its GPU slot for 20 requests before scheduling handoff. Across our full benchmark run, 98.4% of 1.78 B prompt tokens were served directly from cache, reducing prefill to 1.3% of busy time and rendering execution decode-bound (Figure 6).

Deterministic context drainage eliminates hallucinated summaries. Harnesses that compact context typically invoke an LLM to generate summaries (Knutsen and Klein 2026), which costs shared GPU cycles and risks hallucinating persistent false facts into context history. Our context drain operates entirely without model inference: when context exceeds 118k tokens, it deterministically drops the oldest complete turns until history falls below 55k tokens. The system prompt remains pinned, unevicted turns are preserved verbatim, and the symbolic memory store and code graph are re-anchored as a single deterministic block.

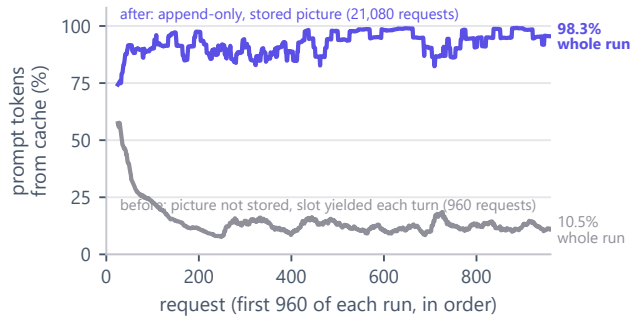


Figure 6 | Share of each request's prompt tokens served from the prefix cache (moving mean over 25 requests), for the first 960 requests of two Qwen3.8-Flash-Next runs on SGLang. Before (19 October, 15 games): the picture is the unstored last message and the GPU slot changes hands every turn. After (23 October, 25 games): every request is the previous request plus its reply, and a game keeps its slot for 20 requests. Labels on the right give the whole run.

Kernel working memory and priority scheduling. The agent maintains four level-keyed symbolic dictionaries in the Python kernel (`GOAL`, `PLAN`, `HYPOTHESES`, `FINDINGS`) that persist across context drains and level transitions. Upon level completion, a curator call consolidates confirmed mechanics into a persistent `SKILLS` repository and registers refuted rules to prevent redundant exploration. Across 25 concurrent games, an asynchronous priority scheduler allocates 10 active GPU slots based on level reward weight discounted by committed actions, multiplying by a decay factor that prevents stalled games from starving progressing ones (Appendix C).

Ablated designs and boundary conditions. Across our development cycle, several architectural alternatives proved detrimental or inert. Replacing past code with semantic stubs caused 6B models to degenerate into copying comment templates in 97.7% of calls, fine-tuning on self-generated successful traces reduced training loss while degrading interactive generalization, and spawning autonomous subagents introduced coordination overhead without performance gain. A complete analysis of these ablated designs and runtime edge cases is provided in Appendix D.

5 Results

Our empirical evaluation establishes that action budget exhaustion on ARC-AGI-3 is driven predominantly by perceptual misinduction rather than downstream planning failure. We first present the controlled experiments and out-of-distribution evaluations of the Perception Engine (Section 5.1), followed by action efficiency and error budget decompositions (Section 5.2), and conclude with hardware-matched comparative benchmarks and systems telemetry (Section 5.3).

Presentation	`ls20` /14	`r11l` /14	Total /28	Int. 1–5	Tokens
Legacy Encoder (baseline)	2	1	3	1	3500
X1 Exploded Inventory	2	1	3	2	180
X2 Surprise Map	3	1	4	2	270
X3 Visual Routines	7	8	15	4	240
X4 Look-alike Threads	4	2	6	3	140
X5 Foresight Strip	4	8	12	4	140
X6 Blueprint	4	4	8	3	100
X7 Role Ledger	11	9	20	5	380
X8 Fused Briefing (Ours)	**13**	**12**	**25**	**5**	**370**

Table 2 | Perception Lab evaluation across candidate presentations. Rubric scores on the opening frames of `ls20` and `r11l`, intuition rating (Int., 1–5), and text token footprint per frame. The Fused Briefing (X8) achieves 25/28 points with an 89.4% reduction in token cost relative to the Legacy Encoder baseline.

5.1 Empirical Evaluation of the Perception Engine

Controlled perception lab on development environments. To evaluate how alternative state representations influence visual scene comprehension, we designed a controlled perception lab evaluating nine distinct presentation formats on the opening frames of `ls20` (discrete arrow-key navigation) and `r11l` (continuous coordinate click interactions). Each presentation was graded against a pre-registered 28-point rubric measuring seven essential perceptual dimensions: agent identification, goal state, mechanical links, hazards, status gauges, terrain composition, and winning plan induction. As detailed in Table 2 and Figure 7, our initial baseline (the Legacy Encoder, `arc_eye`) scored only 3 of 28 points despite consuming 3,500 text tokens per frame. Because it serialized coordinates without relational synthesis, the reasoning model bore the entire burden of deducing spatial semantics, misclassifying functional move gauges as inert screen chrome.

Representational interventions advanced comprehension across two distinct axes. First, host-side geometric computations (X3 Visual Routines and X4 Look-alike Threads) extracted spatial properties that neural vision encoders fail to resolve reliably, such as discovering midpoint tethers and identifying that the `ls20` inventory icon matches the goal glyph rotated by 90° and rescaled. Second, candidate role annotations (X7 Role Ledger) provided rapid operational context, raising the rubric score to 20/28. Synthesizing these approaches into the Fused Briefing (X8) achieved 25 of 28 points at 370 tokens per frame, representing an $8.3\times$ score improvement over the baseline alongside an 89.4% reduction in token consumption. Dumping raw 64×64 ASCII grids alongside per-cell component tables forces the tokenizer to spend $\approx 3{,}500$ tokens per frame encoding repetitive background coordinates—by abstracting connected components on the host, moving spatial layout into the visual Blueprint raster, and emitting only non-redundant geometric relations

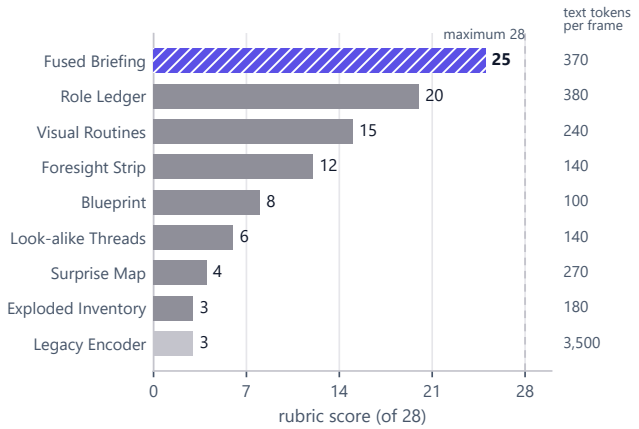


Figure 7 | Perception Lab rubric score versus token cost. The Fused Briefing establishes an empirical Pareto frontier, scoring 25/28 against 3/28 for the Legacy Encoder while reducing prompt tokens by nearly an order of magnitude.

Check on 23 Held-Out Official Games	Empirical Generalization
Screenshot to grid, exact cells	23/23 games (100%)
Agent identity (X7), keyboard games	6/14 games (43%)
Moves-left gauge found (X7)	10/20 games (50%)
Click handles that respond (X7)	7/29 handles (24%)
Keyboard foresight fully right (X5)	1/16 games (6%)

Table 3 | Empirical validation of the Measured–Guessed Rule across 23 held-out environments. Deterministic coordinate recovery and geometric relations generalize unconditionally (100%), whereas latent affordance roles and predictive forward models from a single frame achieve $\leq 50\%$ accuracy, confirming that operational affordances must be framed as testable hypotheses.

in text, the Fused Briefing compresses prompt footprint by 89.4% (to 370 tokens) while increasing prefix-cache capacity by $9.5\times$.

Cross-distribution validation across 23 held-out games. Rubric scoring on the two development frames was conducted non-blind against a pre-registered answer key—out-of-distribution generalization is therefore verified programmatically across the 23 held-out games in Table 3. Ground-truth verification was executed programmatically against the official game engine without manual intervention. As reported in Table 3, deterministic coordinate reconstruction achieved 100% cell accuracy across all 23 games (23/23). Furthermore, computed geometric invariants (4-connected segmentation, bounding enclosures, and dihedral symmetries) proved unconditionally invariant across novel layouts.

In stark contrast, single-frame operational guesses generalized poorly out of distribution. Candidate agent identifications were accurate on only 6 of 14 keyboard environments, moves-left gauge heuristics succeeded on 10 of 20 games,

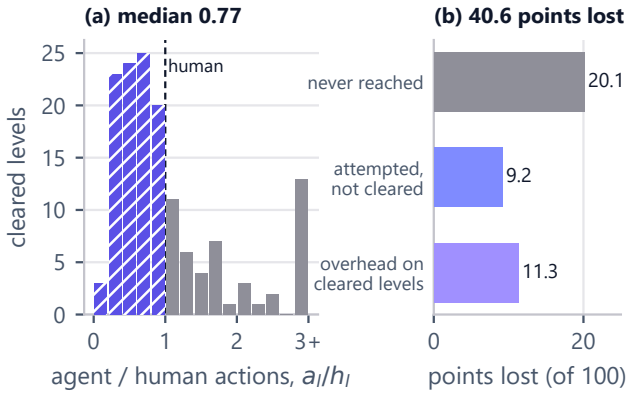


Figure 8 | Action efficiency and score error budget. (a) Distribution of agent-to-human action ratios (a_l/h_l) across 143 cleared levels, where hatched bars indicate performance surpassing human baselines. (b) Decomposition of the 40.6 total points lost from a perfect score, illustrating that 72% of lost credit stems from uncleared levels stalled by perceptual misinduction rather than action overhead on resolved levels.

and click-affordance predictions held on only 7 of 29 candidate handles. Forward predictive simulation (X5) succeeded on only a single game out of 16. These empirical findings substantiate the **Measured–Guessed Rule**: mathematical invariants hold unconditionally by construction and should be asserted as ground truth, whereas latent operational affordances cannot be reliably deduced from static visual frames and must be framed as provisional hypotheses paired with unit-action verification probes.

5.2 Action Efficiency and Error Decomposition

Super-human action efficiency on resolved environments. On levels where the agent successfully induces environment mechanics, SARBLOH executes trajectories with high efficiency. Across 143 cleared levels in the 25 public games, the median ratio of agent actions to the human baseline (a_l/h_l) was **0.77**, with 66% of cleared levels matching or outperforming the human baseline (Figure 8a). Under the RHAE metric, 62% of completed levels earned the maximal score cap of 115%. By synthesizing complete algorithmic search routines in Python and committing multi-step paths via `await play()`, the agent avoids incremental trial-and-error once the scene structure is understood.

Perceptual misinduction dominates the error budget. We decomposed the 40.6 total points lost from a perfect benchmark score using official RHAE weighting (Figure 8b). Unreached levels resulting from earlier stalls accounted for 20.1 lost points, initial uncleared levels contributed 9.2 points, and sub-optimal action overhead on cleared levels represented only 11.3 points. Consequently, 72% of all lost points (29.3 of 40.6) occurred on environments the agent failed to resolve. Trace autopsies on these stalled environments reveal that failure was driven by perceptual misinduction rather than planning depth. On stalled levels, the agent

reformulated its stated goal a median of 8 times (compared to once on cleared levels) and formulated incorrect inductive beliefs, such as declaring `vc33` unsolvable after 739 actions before squandering 1,240 further moves. Misperceiving latent rules blinds the search space, reinforcing that perceptual grounding is the governing operational bottleneck.

5.3 Benchmark Trajectory and Comparative Evaluation

Official held-out Kaggle evaluation trajectory. Because the official Kaggle benchmark executes offline on undisclosed environments, it provides an uncontaminated measure of out-of-distribution reasoning (Figure 9). Initial baselines adapting Prime Agent with open weights scored between 0.00 and 0.27, repeatedly executing voluntary resets without finding a goal. Systematic architectural upgrades advanced performance to 5.14 through SGLang speculative decoding, 5.65 via unified `await play()` programmatic execution, 6.06 through deterministic context drainage, and 10.98 with persistent symbolic memory dictionaries. Integrating the full deterministic perception engine alongside refined drainage established our peak verified score of **16.53**, representing a $13.7\times$ efficiency improvement over the first Milestone winner.

Hardware-matched comparative baseline. To isolate algorithmic contributions from compute resources, we evaluated SARBLOH directly against the first Milestone winner (the Duck harness (Tufa Labs 2026)) on identical server hardware (one NVIDIA RTX PRO 6000) and identical Qwen3.8-Flash-Next model weights. While the baseline ran four parallel passes per game (100 concurrent streams), SARBLOH executed a single scheduled pass across the 25 public games. As shown in Figure 10, SARBLOH consistently accelerated discovery, clearing 73 levels versus 54 at one hour, 97 versus 63 at two hours, and finishing at 143 of 183 levels in 8.2 hours versus 129 distinct levels for the baseline. This efficiency stems from our priority scheduler concentrating compute on game states with high marginal progress potential rather than partitioning resources uniformly.

Comparison with frontier model baselines and systems telemetry. On public environments, SARBLOH achieves an aggregate mean score of 72.5 across evaluation trials, outperforming proprietary frontier models evaluated via unconstrained APIs on 21 of 25 games and achieving 100% resolution on 13 games (Figure 1). Under a single-pass regime, SARBLOH achieves a mean score of 59.4, exceeding the strongest frontier baseline (GPT-6.1 Sol at 54.8). In terms of runtime telemetry on our single RTX PRO 6000 GPU, the append-only context preserved a 98.4% prompt prefix-cache hit rate across 1.78 B total prompt tokens, ensuring that prefill consumed only 1.3% of busy execution time compared to 50.1% for decode generation. Across 21,224 model calls, the runtime suffered zero server failures. Runtime failure modes and ablated designs are detailed in Appendix D.

6 Related Work

Perceptual representations in interactive ARC agents. Prior ARC-AGI-3 architectures differ across planning strategies, context memory management, and external tooling, yet they share a uniform perceptual assumption: the visual grid is transmitted directly to the reasoning model as raw unparsed data, delegating spatial scene decomposition entirely to neural inference (Table 4). Specifically, the first Milestone winner (the Duck harness (Tufa Labs 2026; ARC Prize Foundation 2026b)) serializes the board as composite ASCII character matrices alongside rendered image frames. The second- and third-place Kaggle systems pass annotated screenshots to Gemma-4-31B, prompting the vision model to emit JSON action sequences directly (ARC Prize Foundation 2026b). In the frontier API regime, VISTA (Han et al. 2026) deploys a multimodal harness with lossless visual frame retention, while Arcgentica (Knutsen and Klein 2026) and Prime Agent (Karten et al. 2026) expose the environment within a persistent Python REPL, leaving the language model to write its own parsing scripts over raw coordinate arrays.

Core knowledge grounding and geometric invariants. Across all existing systems, the agent observes the world as an integer matrix, an ASCII string, or an unannotated visual raster. Consequently, fundamental spatial primitives, including 4-connected entity segmentation, topological containment, lattice-constrained reachability, and affine symmetries, are left to be reconstructed by the neural backbone from raw tokens. As our motivating ablations establish (Section 3), this unparsed representation forms the primary source of action inefficiency and hallucination. In contrast, SARBLOH computes geometric invariants deterministically on the host in $\mathcal{O}(HW)$ time, maintains persistent bipartite entity tracking across temporal steps, and explicitly decouples mathematical certainties from single-action testable affordance hypotheses.

Autonomous code generation and programmatic execution. The CodeAct paradigm (Wang et al. 2024) demonstrated that expressing agent actions through executable programming primitives outperforms specialized JSON action schemas. Prime Agent (Karten et al. 2026) adapted this to ARC-AGI-3 by allowing models to execute Python code within an interactive environment. However, earlier harnesses decoupled model reasoning into disjoint tools for code simulation and action execution, forcing expensive context-switching roundtrips. SARBLOH unifies these operations within an asynchronous execution contract (`await play()`), allowing models to hypothesize, simulate, and commit multi-step actuation trajectories within a single reasoning turn.

7 Limitations and Future Work

Limitations. Several architectural limitations remain. First, while affordance hypotheses are coupled with unit falsification tests, the current perception engine evaluates hypotheses independently at each frame without maintaining a fully unified Bayesian belief distribution across multi-step episodes. Second, although the agent plans with super-human efficiency once a level is understood, antecedent failure on complex perceptual transitions remains the primary source of score degradation. Finally, our current results rely on general-purpose open-weight models without task-specific policy fine-tuning.

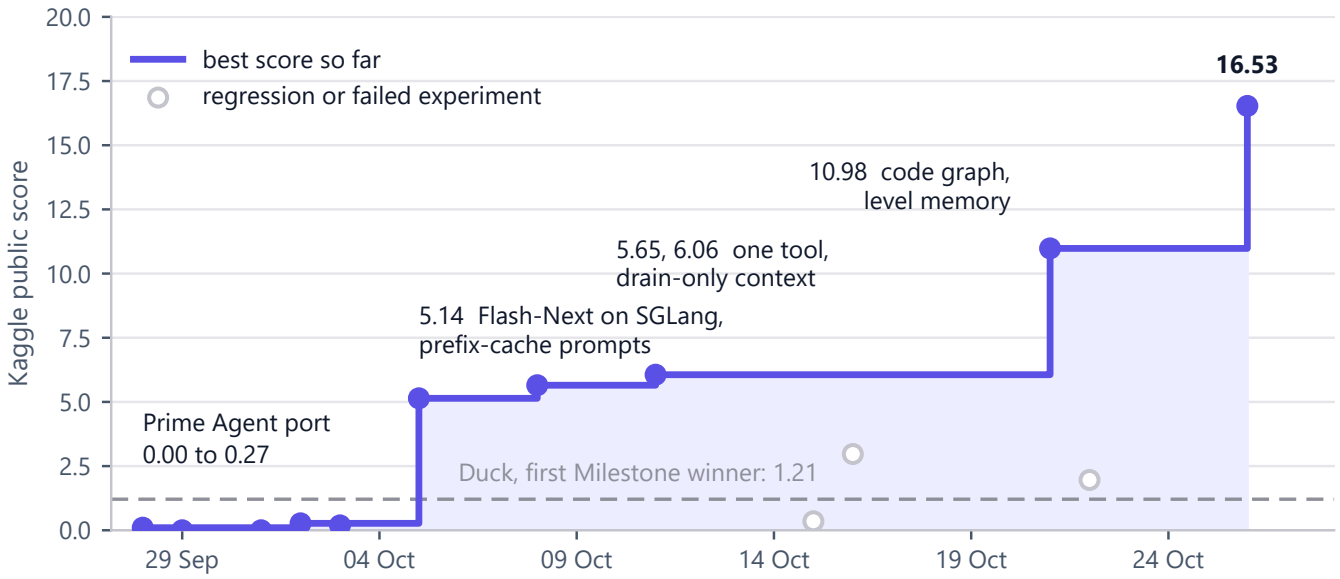


Figure 9 | Evaluation trajectory across completed submissions on the official held-out Kaggle benchmark. The solid curve tracks cumulative peak efficiency score, hollow markers indicate regressive exploratory trials, and annotations denote primary architectural interventions.

Future work. Promising research avenues include:

1. **Closed-Loop Belief Updating**: Formulating affordance induction as a dynamic partially observable belief state, where the empirical outcomes of unit falsification probes explicitly update prior belief weights across compositional levels.
2. **Offline Trajectory Fine-Tuning**: Applying parameter-efficient fine-tuning (LoRA (Hu et al. 2021)) directly on our database of verified, high-efficiency execution trajectories, optimizing syntactic code generation and world model synthesis.
3. **Sample-Efficient Reinforcement Learning**: Investigating offline reinforcement learning algorithms, such as Group Relative Policy Optimization (GRPO (Shao et al. 2024)) and balanced policy clipping, to optimize planning depth and action parsimony within our fixed computational envelope.

8 Conclusion

Autonomous reasoning in uninstructed interactive environments is fundamentally constrained by perception rather than downstream algorithmic planning. Under the non-linear action penalties of ARC-AGI-3, agents cannot afford to compensate for impoverished representations through exploratory trial and error. As our empirical investigations demonstrate, presenting environments as unparsed coordinate matrices induces perceptual misinduction, compelling language and vision backbones to squander their action budgets attempting to reconstruct topological adjacency and basic spatial mechanics.

To resolve this limitation, we introduced Sarbloh, an autonomous agent architecture organized around the Measured–Guessed Rule. By computing deterministic geometric invariants directly on the host in linear time, the perception engine extracts lattice reachability, topological enclosures, midpoint tethers, and dihedral symmetries as mathematical certainties that generalize unconditionally across novel environments. Simultaneously, the framework treats latent operational semantics not as brittle classifications, but as provisional inductive hypotheses paired with minimal unit-action verification probes. Grounded by this perceptual foundation, the agent models environment transitions through programmatic Python execution, dispatching multi-step actuation trajectories that halt dynamically upon encountering unexpected state transitions.

Our findings indicate that advancing fluid intelligence on novel benchmarks does not strictly necessitate larger parameter scales or unconstrained inference budgets. Instead, equipping reasoning models with structured, core-knowledge perceptual abstractions bridges the divide between visual observation and symbolic deduction, enabling open-weight architectures to explore with disciplined, sample-efficient physical agency.

	Hidden Kaggle set (offline, 1 GPU, 9 h)			Public set (frontier APIs, unbounded)				
	Sarbloh this work	**Duck** first Milestone winner	**Reki** second Milestone winner	**Prime Agent** open harness	**Arcgentica** Symbolica	**AVO** NVIDIA	**Provider Adapters** GPT-5.5	**Provider Adapters** Gemini 3.8 Flash
Backbone	Qwen3.8-Flash-Next (MoE)	Qwen 3.6 27B	Gemma-4-31B	Claude Opus 5	Claude Opus 4.6	Opus 5 / GPT-5.6 Sol	GPT-5.5	Gemini 3.8 Flash
Open weights	✓	✓	✓	–	–	–	–	–
Score	**16.53**	**1.21**	**0.87**	**95.5**	**36.1**†	**100.0**†	**99.99**†	≈**99.9**†

Table 4 | ARC-AGI-3 systems across two distinct evaluation regimes. The left group runs offline on hidden Kaggle environments with open-weight models on a single GPU; the right group evaluates frontier models via unconstrained commercial APIs on public environments. †Self-reported by authors on public games without independent certification.

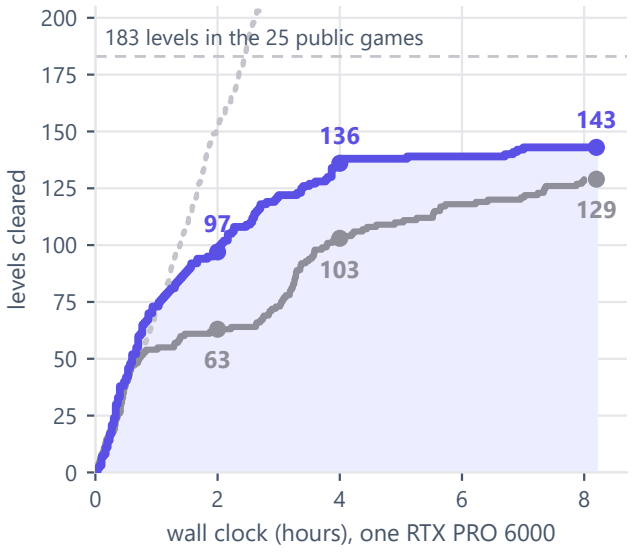


Figure 10 | Cleared levels over wall-clock execution time on 25 public environments under identical hardware. Indigo: Sarbloh (single pass). Solid grey: distinct levels cleared by the first Milestone winner (Duck harness) across four parallel passes; dotted grey: total levels summed across passes. Sarbloh clears 143 of 183 levels in 8.2 hours, surpassing the baseline's 129 distinct levels.

References

ARC Prize Foundation. 2026a. ARC-AGI-3: A New Challenge for Frontier Agentic Intelligence. *arXiv preprint arXiv:2603.24621*.

ARC Prize Foundation. 2026b. ARC Prize 2026: ARC-AGI-3 Milestone #1 Results. https://arcprize.org/blog/arc-prize-2026-milestone-1.

Cheng, W.; Zhang, W.; Shen, H.; Cai, Y.; He, X.; et al. 2023. Optimize Weight Rounding via Signed Gradient Descent for the Quantization of LLMs. *arXiv preprint arXiv:2309.05516*.

Chollet, F. 2019. On the Measure of Intelligence. *arXiv preprint arXiv:1911.01547*.

Han, Q.; Hu, K.; Qiu, L.; Wu, C.; and He, K. 2026. VISTA: A Visual Harness for Reasoning in an Interactive World. https://vista-research.github.io/.

Hu, E. J.; Shen, Y.; Wallis, P.; Allen-Zhu, Z.; Li, Y.; et al. 2021. LoRA: Low-Rank Adaptation of Large Language Models. *arXiv preprint arXiv:2106.09685*.

Karten, S.; Zhang, A. L.; Thomas, K.; Müller, S.; Bakouch, E.; et al. 2026. Prime Agent: A Self-Improving RLM Harness. *arXiv preprint arXiv:2608.23552*.

Knutsen, S.; and Klein, V. 2026. Arcgentica: ARC-AGI-3 Agent Harness Built on the Agentica SDK. https://github.com/symbolica-ai/ARC-AGI-3-Agents.

Li, Y.; Wei, F.; Zhang, C.; and Zhang, H. 2024. EAGLE: Speculative Sampling Requires Rethinking Feature Uncertainty. *arXiv preprint arXiv:2401.15077*.

Marr, D. 1982. *Vision: A Computational Investigation into the Human Representation and Processing of Visual Information*. W. H. Freeman.

Rahmanzadehgervi, P.; Bolton, L.; Taesiri, M. R.; and Nguyen, A. T. 2024. Vision Language Models Are Blind: Failing to Translate Detailed Visual Features into Words. *arXiv preprint arXiv:2407.06581*.

Shao, Z.; Wang, P.; Zhu, Q.; Xu, R.; Song, J.; et al. 2024. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. *arXiv preprint arXiv:2402.03300*.

Spelke, E. S.; and Kinzler, K. D. 2007. Core Knowledge. *Developmental Science*, 10(1): 89–96.

Tufa Labs. 2026. The Duck: Tufa Labs' ARC-AGI-3 Harness, Milestone #1 Winner. Kaggle notebook, https://www.kaggle.com/code/jeroencottaar/tufa-labs-duck-harness-june-30-milestone-winner.

Ullman, S. 1984. Visual Routines. *Cognition*, 18(1-3): 97–159.

Wang, X.; Chen, Y.; Yuan, L.; Zhang, Y.; Li, Y.; et al. 2024. Executable Code Actions Elicit Better LLM Agents. *arXiv preprint arXiv:2402.01030*.

Yang, S.; Kautz, J.; and Hatamizadeh, A. 2024. Gated Delta Networks: Improving Mamba2 with Delta Rule. *arXiv preprint arXiv:2412.06464*.

Zheng, L.; Yin, L.; Xie, Z.; Sun, C.; Huang, J.; et al. 2023. SGLang: Efficient Execution of Structured Language Model Programs. *arXiv preprint arXiv:2312.07104*.

A Perception Lab: Experimental Protocols and Presentations

This appendix documents the experimental protocols and candidate presentations developed during the Perception Lab. Each experiment was evaluated as a self-contained module taking a raw game frame and synthesizing an image and textual representation for the reasoning model. The primary quantitative results, including the 28-point rubric comparison and 23-game held-out validation, are presented in Section 5.1 (Table 2, Figure 7, and Table 3).

A.1 Setup and Answer Key

Shared front end. All experimental arms (X1–X8) share an identical front end to isolate representational formatting from extraction noise. A raw frame is transformed into an exact 64×64 categorical color grid by fitting cell pitch and origin from chromatic boundary gradients. The grid is segmented into background terrain (modal color coverage exceeding 8%), four-connected monochromatic entities categorized by morphology (compact objects, icons, dotted structures, thin connectors, or bars), bounding panels, and peripheral heads-up display (HUD) elements within three cells of the screen boundary.

Baseline encoder. The baseline is our legacy text-only encoder (`arc_eye`). It extracts the fitted lattice, assigns a fixed palette symbol per cell, performs connected component segmentation with hole detection, computes dihedral shape hashes, and identifies bounding boxes alongside inter-frame coordinate differences. It emits an exhaustive text dump requiring approximately 3,500 tokens per frame without visual images.

Evaluation rubrics. The evaluation suite comprises the opening levels of `ls20` (arrow-key lattice navigation) and `r11l` (continuous coordinate click interactions). Prior to running any experimental arm, an objective answer key was formulated across seven core items: K1 controllable entity identity, K2 target goal state, K3 structural link or carried object, K4 modifier or hazard, K5 status gauges, K6 winning trajectory plan, and K7 terrain boundaries. Each item was scored: 2 points for unmistakable and accurate description, 1 point for partial description, 0 for omitted information, and -1 for actively misleading inductions, yielding a maximum score of 14 points per environment (28 points aggregate). An intuition rating from 1 to 5 was also recorded alongside prompt token expenditure.

A.2 Candidate Presentations

X1 Exploded Inventory. Entities are de-duplicated by exact shape grids, magnified to a standard display scale, arranged spatially with letter tags, and annotated with multiplicity counts ($\times N$). A miniature thumbnail outlines original entity positions. Thin connectors are cataloged by endpoint coordinates.

X2 Surprise Map. Calculates cell-level information surprise as $-\log_2 p$ based on 3×3 neighborhood pattern distributions. Periodic floor tiles and boundary walls score near zero bits. Entities are ranked by total bits, preserving full saturation only on high-surprise components while fading predictable background textures.

X3 Visual Routines. Executes serial geometric routines based on Ullman (1984). **Reach** performs breadth-first search across detected lattice steps, reporting path lengths from the likely agent to all visible targets. **Fit** correlates hollow enclosing boundaries with objects of identical dimensions. **Trace** tracks single-pixel connectors to their endpoints, identifying entities suspended at the midpoint of dual anchors. **Align** catalogs entities sharing identical row or column spans.

X4 Look-alike Threads. Compares binary entity shapes under all eight dihedral symmetries of the square alongside integer scaling factors ($D_4 \times \mathbb{Z}^+$). Matched pairs are classified as EXACT (identical copies), TURNED (matched under rotation or reflection), NEAR (differing by at most three cells), or STATE (identical geometry with modified color palettes). Visual threads connect matched entities across the frame.

X5 Foresight Strip. Projects prospective states for legal actions using hardcoded physical priors: agent sprites translate by their bounding dimensions until striking walls, edge gauges decrement by unit length per action, and midpoint objects track anchor displacements.

X6 Blueprint. Re-renders the scene as an architectural schematic following Marr (1982): terrain rendered in soft tones, structural boundaries hatched, tile lattices faintly gridded, and HUD elements isolated into a dedicated side panel.

X7 Role Ledger. Scores non-terrain entities against eight standard gameplay roles (YOU, GOAL, MOVES LEFT, LIVES, INVENTORY, MODIFIER, HANDLE, CARRIED THING) using evidence heuristics, printing each top candidate alongside an empirical unit-action verification test.

X8 Fused Briefing. Synthesizes the architectural blueprint canvas of X6, the relational threads and midpoint indicators of X3–X4, and a two-tiered text briefing explicitly separating MEASURED geometric certainties from GUESSED affordance hypotheses paired with unit-action verification tests.

Figure 11 provides a visual comparison of each candidate presentation evaluated on `ls20`.

B Perception Engine Structural Briefings

Below we provide the verbatim structured briefing text emitted by the deterministic perception engine for the opening frames of `ls20` and `r11l` illustrated in Figure 5. The object identifiers correspond exactly with the annotated spatial map and the kernel's symbolic state dictionary (`observe().objects`).

```
BRIEFING (arrow keys)
MEASURED (certain):
- Floor is a 5x5 tile grid.
- obj 4 (blue 6-cell shape): sits inside a black box
     that is NOT plain floor; the floor reaches the box's
      bottom side in 5 steps (the only way in)
- obj 6 (white/light grey plus): reachable in 6 steps
```

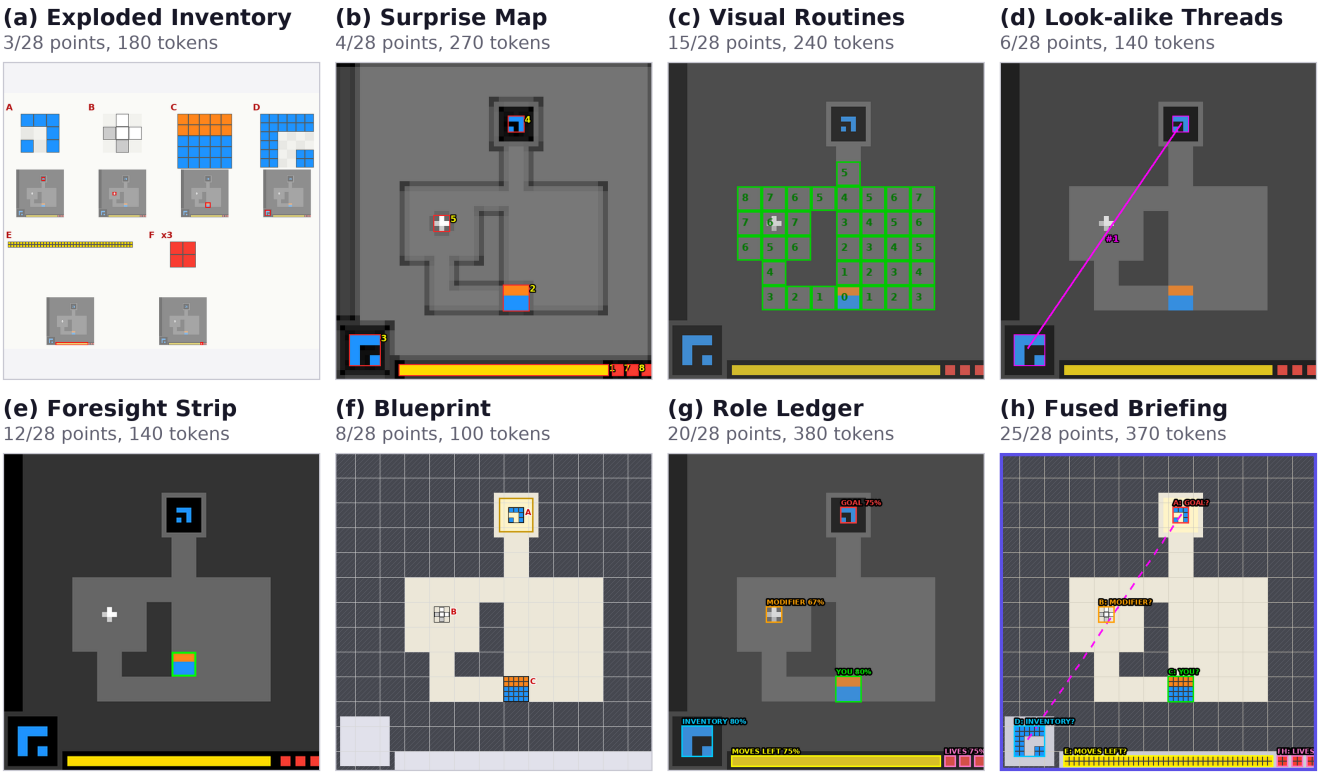


Figure 11 | Visual presentations evaluated in the Perception Lab on `ls20`. (a) Exploded Inventory separates unique entity morphologies. (b) Surprise Map highlights unpredicted local textures. (c) Visual Routines computes lattice BFS step counts to candidate targets. (d) Look-alike Threads connects shapes matching under rotation, reflection, or integer scaling. (e) Foresight Strip simulates counterfactual forward states. (f) Blueprint redraws terrain and lattices as a clean architectural schematic. (g) Role Ledger annotates candidate affordance hypotheses. (h) Fused Briefing combines the blueprint map with measured invariants and testable role hypotheses.

```
- obj 4 (blue 6-cell shape) is directly above obj 10 (
    blue/orange square) (shared columns 35-37)
- obj 12 (blue irregular shape (24 cells)) (drawn 2x
    bigger), mirrored left-right OR turned 90 deg
    clockwise, becomes exactly obj 4 (blue 6-cell shape)
- 3 identical red square (2x2): obj 16, 17, 18 -> a
    count of something (lives, items, tiles)
- obj 6 (white/light grey plus) has a symmetric shape
    but lopsided colours (marks a direction/state)
- obj 10 (blue/orange square) has a symmetric shape but
    lopsided colours (marks a direction/state)
GUESSES (check with one action):
- obj 10 (blue/orange square) = YOU (80%) - test: press
    one arrow key: it should move one tile and the gauge
     should drop
- obj 12 (blue irregular shape (24 cells)) = INVENTORY
    (80%) - test: watch it after touching the modifier
- obj 4 (blue 6-cell shape) = GOAL (75%) - test: get the
     right thing into it and see if the level ends
- obj 15 (yellow bar) = MOVES LEFT (75%) - test: take
    any action and check it shrinks by one unit
- obj 16, 17, 18 (red square) = LIVES (75%) - test:
    watch it when the gauge runs out or you fail
- obj 6 (white/light grey plus) = MODIFIER (67%) - test:
     step onto it, then check whether the inventory icon
      changed


BRIEFING (clicks)
MEASURED (certain):
- no tile grid -> free movement. Open space = black (81%
     of screen).
- all 4 main objects share one open space; straight
    paths between them are ALL CLEAR of rock
- the grey areas are not open floor (likely walls)
- obj 4 (purple dotted ring) encloses an EMPTY 5x5 space
     -- exactly the size of: obj 16 (white/purple
    diamond), obj 25 (purple/magenta disc), obj 36 (dark
     grey/purple diamond)
- a light grey line connects obj 16 (white/purple
    diamond) to obj 25 (purple/magenta disc)
- a light grey line connects obj 25 (purple/magenta disc
    ) to obj 36 (dark grey/purple diamond)
- obj 25 (purple/magenta disc) sits at the MIDPOINT of
    obj 16 (white/purple diamond) and obj 36 (dark grey/
    purple diamond) (error 0.5 cells) -> it is held
    between them; moving either end should move it to
    the new midpoint
- obj 16, 36 (white/purple diamond; dark grey/purple
    diamond) have the same shape in different colours (
```

```
    white/purple vs dark grey/purple)
GUESSES (check with one action):
- obj 4 (purple dotted ring) = GOAL (80%) - test: get
    the right thing into it and see if the level ends
- obj 16, 36 (white/purple diamond; dark grey/purple
    diamond) = HANDLE (click) (80%) - test: click an
    empty spot once: the selected handle should jump
    there
- obj 1 (white bar) = MOVES LEFT (75%) - test: take any
    action and check it shrinks by one unit
- obj 25 (purple/magenta disc) = CARRIED THING (75%) -
    test: move one handle: it should slide to the new
    midpoint
```

C System Overview and Priority Scheduler

Figure 12 illustrates the architectural topology. The priority scheduler allocates GPU execution slots to game instances based on expected marginal score utility. For a game currently attempting level $L+1$ of n levels, having expended t generation tokens and a committed physical actions on the current attempt without a solution, priority π is formulated as:

$$\pi = \big(g_{\text{now}} + g_{\text{tail}}\big) \cdot \underbrace{2^{-\left((t/T)^2+(a/A)^2\right)}}_{\text{hope}}, \tag{4}$$

$$g_{\text{now}} = \frac{L+1}{n(n+1)/2}\Big(\frac{h}{h+a}\Big)^2, \quad g_{\text{tail}} = \frac{\tau_{\text{tail}}(n-L-1)}{55}\,\phi,$$

where g_{now} reflects the RHAE reward weight of the active level discounted quadratically by committed actions, and g_{tail} estimates the expected option value of unlocking subsequent compositional levels. We set $h = 25$ actions (representative of human level completion counts), $T = 40$k tokens, and $A = 115$ actions. The tail function $\tau_{\text{tail}}(k)$ evaluates to 8, 7, 5, or 0 when $k \geq 3$, 2, 1, or 0 levels remain unreached, normalized on the 55-weight triangular scale of a 10-level game to prevent short games from starving complex ones. The temporal factor $\phi \in [0, 1]$ decays tail value linearly over the final 20% of the nine-hour execution budget, re-focusing compute on resolving active levels rather than unlocking future ones. Games that have never held a GPU slot receive strict initial priority, and active games retain their slot for a 20-request quantum before scheduling re-evaluation. All hyperparameter constants were set empirically without formal grid searches.

D Negative Results, Ablated Designs, and Failure Modes

This section documents architectural designs, runtime edge cases, and engineering interventions that failed to yield performance improvements or introduced instability during live benchmark evaluation.

D.1 Perceptual Edge Cases and Line Budget Quotas

Lattice detection instability from peripheral HUD elements. In early iterations, the host-side lattice detection module fitted tile dimensions over all non-terrain pixels. In `ls20`, when the player moved, an off-grid gauge within the status box introduced irregular coordinate intervals, causing the fitted 5×5 grid to collapse. Similarly, in `ka59`, touching a secondary sprite altered the detected lattice periodicity. Each collapse temporarily deleted all reachability invariants from context, causing the agent to vacillate between conflicting spatial hypotheses. We resolved this by explicitly excluding peripheral HUD regions from the lattice fitting routine and caching the initial valid lattice configuration for the remainder of the level.

Combinatorial relation crowding. In complex environments with dense entity distributions, pairwise coordinate relations expand quadratically. In `tr87`, the initial frame produces 60 alignment pairs alongside 7 enclosure relations and 1 symmetry relation, while `bp35` yields 792 alignment pairs. Under a naive global line cap, high-frequency coordinate alignments saturate the briefing, crowding out vital invariants such as dihedral symmetries and candidate affordance roles (Figure 13a). Under our proportional sharing policy, line quotas are allocated across invariant categories, reserving up to one-third of the display for candidate role hypotheses and summarizing remaining relations as aggregate counts (Figure 13b).

D.2 Rejected Architectural Designs

Sparse delta state encoding. Transmitting only differential coordinate changes between frames substantially reduces prompt token consumption. However, in ARC-AGI-3 environments where local actions trigger distant causal effects (e.g., activating a floor plate updating an icon across the display), sparse updates conceal non-local covariance. In development traces, models receiving delta updates consistently concluded that actions were non-functional when localized pixels remained static, leading to catastrophic action loops. Full-frame visual representations remain essential for maintaining causal coherence.

Disjoint tool spaces for reasoning and actuation. Providing separate tool interfaces for code execution and environment stepping caused models to fragment reasoning trajectories. The agent formulated world models in code, exited the REPL to dispatch individual actions, and re-entered code execution to reconstruct state variables, multiplying inference latency. Merging action commitment directly into the Python kernel via `await play()` was the single intervention responsible for advancing the hidden-set score from 5.14 to 5.65.

Semantic history comment stubbing. To conserve context window capacity, we tested replacing executed historical code cells with brief descriptive comments. Across two test runs on 16 and 25 October, reasoning backbones degenerated into copying comment strings rather than generating executable Python code (occurring in 97.7% of turns and clearing 0 of 97 levels). Smaller open-weight models heavily

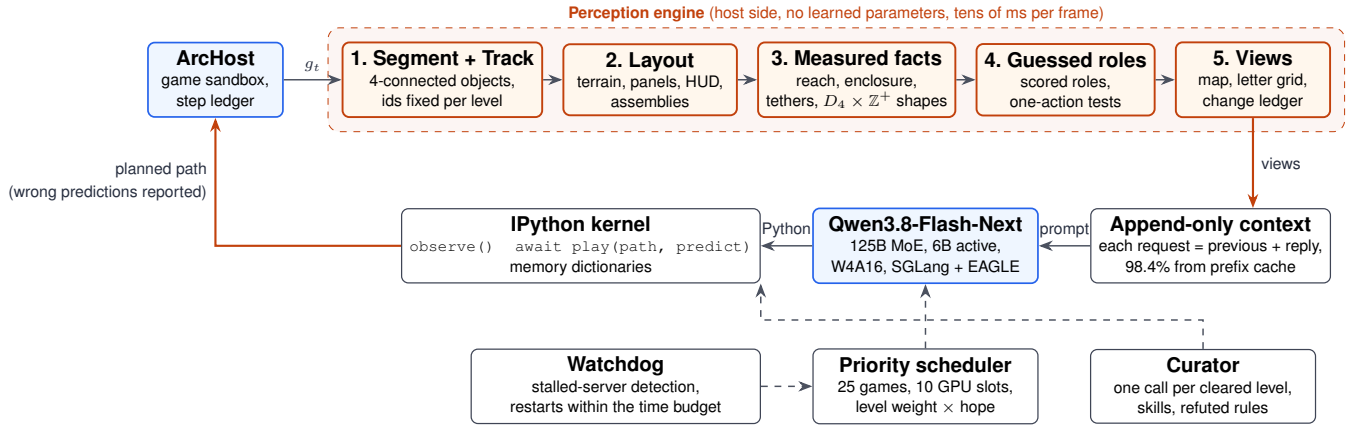


Figure 12 | System topology of SARBLOH. The host-side perception engine extracts invariant relations and affordance hypotheses from each frame, passing views to an append-only context. The reasoning model generates Python search routines in a persistent kernel, committing multi-step paths via `await play()`. Twenty-five games share ten GPU execution slots under the utility-driven priority scheduler.

imitate local context formatting; historical code must remain intact as functional syntax.

Autonomous subagent hierarchies. Providing the model with programmatic subagent delegation primitives (following the Prime Agent specification) yielded zero productive invocations across all benchmark runs. Open-weight 6B-active models failed to effectively coordinate subordinate contexts, adding prompt complexity without empirical utility.

Self-imitation fine-tuning on cleared trajectories. We fine-tuned Qwen3.8-Flash-Next on successful execution traces using low-rank adaptation (LoRA). While training cross-entropy loss decreased from 11% to 6%, out-of-distribution benchmark performance degraded significantly. The fine-tuned weights exhibited rigid memorization of specific coordinate paths rather than generalized spatial reasoning, consuming over 80% of our training compute budget without positive yield.

D.3 Interactive Runtime Failure Modes

Degenerate holding loops under low-budget prompt regimes. In an extended 25-game run on 23 October, four game instances (`lf52`, `s5i5`, `vc33`, `re86`) halted within 15 actions of their 2,000-action ceiling. When prompted to continue, the model generated repetitive "`Holding...`" responses across 14,000 consecutive turns, consuming 1.24 B prompt tokens without committing actions. While these stalls occurred after all solvable levels had been completed and thus did not reduce final score, they needlessly consumed wall-clock time. Enforcing an explicit ceiling of 400 execution turns per game eliminated this failure mode.

E System Configuration and Hyperparameters

Table 5 documents the primary execution parameters governing the scored benchmark evaluation.

System Configuration Parameter	Operational Value
Concurrent games / Active GPU slots	25 games / 10 slots
Scheduler quantum allocation	20 model calls per round
Action allocation ceiling per game	2,000 actions
Maximal steps per `play()` path	$\leq$100 actions
Turn reply budget / Inner requests	12k tokens / 12 calls
Turn preemption thinking threshold	16k tokens
Context drain threshold / target	118k tokens / 55k tokens
Context maximum window size	131,072 tokens
Visual map rendering scale	10 pixels per grid cell
Briefing line limit / Short-view ceiling	20 lines / 600 tokens
Symbolic memory caps (`G`/`P`/`H`/`F`)	150 / 200 / 400 / 300 tokens
Sampling temperature / top-p / top-k	$T = 0.6$ / $p = 0.95$ / $k = 20$
Serving engine and precision	SGLang, W4A16, FP8 KV cache
Speculative decoding configuration	EAGLE (3 speculative draft steps)

Table 5 | System hyperparameter settings deployed in the official evaluation runtime.

F Artifacts and Reproducibility

All empirical findings, telemetry traces, and figures in this manuscript are fully reproducible using the scripts in `Writing/scripts/`:

- `collect_data.py`: Ingests the Kaggle submission log, per-game benchmark tables, and perception lab evaluation rubrics.
- `extract_curves.py`: Extracts wall-clock level completion trajectories from raw execution logs.
- `make_perception_figure.py`: Executes the host-side perception engine on raw game environments to generate segmented objects, spatial maps, and structured



Figure 13 | Briefing line budget allocation on `tr87`. (a) Under an unweighted 30-line cap, 60 pairwise coordinate alignments saturate the observation space, crowding out symmetry invariants and affordance hypotheses. (b) Under proportional line-quota scheduling, alignment relations are bounded, preserving visibility for critical geometric symmetries and candidate affordance tests while reporting surplus relations as aggregate counts.

briefings.

- `plot_results.py`: Generates all publication-grade figures in vector PDF and high-resolution raster formats.

All code, model serving scripts, evaluation environments, and experimental logs are publicly accessible under the MIT-0 license (a permissive public-domain-equivalent license, as the competition rules require) at https://github.com/Harsidak/SARBLOH.